

Methods: Dynamical Model and Control Laws for a 3D Drone Swarm Simulation

1 Overview

This section describes the mathematical formulation, physical modeling assumptions, and control laws used to simulate a three-dimensional coordinated drone swarm. The method couples Newtonian point-mass dynamics with biologically inspired flocking laws and additional mission-oriented control terms. Obstacle avoidance, multi-group coordination, and individual drone behaviors (e.g., loiter, waypoint navigation, rejoining) are included through a unified acceleration-based framework.

The objective is to provide a reproducible, physics-grounded formulation appropriate for use in journal articles and technical documentation.

2 Physical Model

Each drone is modeled as a point mass governed by second-order dynamics. Let $\mathbf{p}_i(t) \in \mathbb{R}^3$ and $\mathbf{v}_i(t) \in \mathbb{R}^3$ denote the position and velocity of drone i , and $\mathbf{a}_i(t)$ its acceleration. The governing equations are:

$$\dot{\mathbf{p}}_i = \mathbf{v}_i, \tag{1}$$

$$\dot{\mathbf{v}}_i = \mathbf{a}_i. \tag{2}$$

The acceleration $\mathbf{a}_i$ is assembled from several terms representing goal attraction, separation, alignment, altitude tracking, obstacle avoidance, damping, and mode-specific behaviors.

All drones share identical mass, which is absorbed into the control gains. Orientation dynamics, aerodynamic forces, and actuator bandwidth limitations are not explicitly modeled; the simulated accelerations represent the ideal control commands.

2.1 Assumptions

The primary simplifying assumptions are:

- **Point-mass kinematics:** Drones have no attitude dynamics; only translational motion is simulated.
- **Instantaneous control response:** Accelerations can be commanded directly, subject to a maximum magnitude $a_{\max}$.

- **All-to-all sensing within groups:** Separation and alignment terms assume each drone can sense the state of all drones in its assigned group.
- **Static obstacles:** Obstacles such as cylinders and walls do not move over time.

3 Control Architecture

The overall acceleration applied to drone i is:

$$\mathbf{a}_i = \mathbf{a}_i^{\text{goal}} + \mathbf{a}_i^{\text{align}} + \mathbf{a}_i^{\text{sep}} + \mathbf{a}_i^{\text{alt}} + \mathbf{a}_i^{\text{obs}} - c_d \mathbf{v}_i, \quad (3)$$

where c_d is a linear damping coefficient. Additional modifications apply depending on drone mode (navigate, loiter, rejoin, landed).

3.1 Goal Attraction

Each drone is driven toward its group goal position $\mathbf{g}$ through a proportional control law:

$$\mathbf{a}_i^{\text{goal}} = k_g(\mathbf{g} - \mathbf{p}_i), \quad (4)$$

where k_g is a gain parameter. For independent navigation, a higher gain $2k_g$ is used to accelerate waypoint convergence.

3.2 Velocity Alignment

To promote coherent group motion, drones align their velocities toward the group mean:

$$\mathbf{a}_i^{\text{align}} = k_a(\bar{\mathbf{v}} - \mathbf{v}_i), \quad (5)$$

where k_a is the alignment gain and $\bar{\mathbf{v}}$ is the average velocity of drones in the same group.

3.3 Separation-Length Regulation

To maintain a desired inter-drone spacing L_s , we include a pairwise spacing term:

$$\mathbf{a} * i^{\text{sep}} = \frac{k_s}{N_g - 1} \sum_{*j \in g, j \neq i} (d_{ij} - L_s) \hat{\mathbf{r}} * ij, \quad (6)$$

where N_g is the number of drones in group g , $d * ij = |\mathbf{p}_j - \mathbf{p} * i|$, and $\hat{\mathbf{r}} * ij = (\mathbf{p}_j - \mathbf{p} * i) / d * ij$. This encourages expansion when drones are too close and contraction when they are too far apart.

3.4 Altitude Tracking

A soft altitude regulation term is included:

$$\mathbf{a} * i^{\text{alt}} = k_h(h^* - p * i, z) \mathbf{e}_z, \quad (7)$$

with h^* the target altitude and $\mathbf{e}_z$ the vertical unit vector.

4 Obstacle Repulsion Model

Obstacles exert repulsive forces when a drone approaches within a safety distance d_{safe} . The general form is:

$$\mathbf{a} * i^{\text{obs}} = f(d_i), \hat{\mathbf{n}} * i, \quad (8)$$

where d_i is the distance from the drone to the obstacle's surface, $\hat{\mathbf{n}} * i$ is the outward surface normal, and the scalar repulsion follows

$$f(d) = k_o \left(\frac{1}{d} - \frac{1}{d * \text{safe}} \right), \quad f = 0 \text{ if } d \geq d * \text{safe}, \quad (9)$$

with the magnitude capped at $f * \text{max}$.

4.1 Cylindrical Obstacles

A cylinder is defined by center (c_x, c_y) , radius R , and vertical bounds z_0, z_1 . For each drone, we compute the closest point on either the lateral surface or the top/bottom caps. The repulsion uses the nearer of the two.

4.2 Planar Walls

Walls are rectangular panels defined by a center point, two in-plane unit vectors $(\mathbf{u}, \mathbf{v})$, a normal vector $\mathbf{n}$, and half-widths (w, h, t) . The closest point is found by projecting the drone into the local wall frame and clamping the coordinates to the panel extents.

5 Mode-Specific Control Laws

5.1 Waypoint Navigation

For independent flight, drone i navigates to waypoint $\mathbf{w}$ using

$$\mathbf{a}_i = 2k_g(\mathbf{w} - \mathbf{p}_i) - 1.5c_d\mathbf{v}_i + \mathbf{a}_i^{\text{obs}}. \quad (10)$$

5.2 Loiter Behavior

Loitering holds the drone at a fixed position $\mathbf{p} * \ell$:

$$\mathbf{a} * i = k * \ell(\mathbf{p} * \ell - \mathbf{p}_i) - 3c_d\mathbf{v} * i, \quad (11)$$

with $k * \ell$ a strong hovering gain.

5.3 Rejoining Behavior

Rejoining blends independent navigation and swarm behavior:

$$\mathbf{a}_i = (1 - \beta_i)\mathbf{a}_i^{\text{nav}} + \beta_i\mathbf{a}_i^{\text{swarm}}, \quad (12)$$

where $0 \leq \beta_i \leq 1$ increases over time until the drone fully reintegrates.

6 Acceleration Saturation

To maintain physical plausibility, accelerations satisfy

$$|\mathbf{a} * i| \leq a * \max, \quad (13)$$

with vector clipping applied when necessary.

7 Time Integration

The dynamics are integrated using the classical fourth-order Runge–Kutta (RK4) scheme with constant timestep Δt :

$$\mathbf{k}_1 = f(\mathbf{p}, \mathbf{v}), \mathbf{k}_2 = f\left(\mathbf{p} + \frac{1}{2}\Delta t, \mathbf{k}_1, \mathbf{v} + \frac{1}{2}\Delta t, \mathbf{k}_1\right), \mathbf{k}_3 = f\left(\mathbf{p} + \frac{1}{2}\Delta t, \mathbf{k}_2, \mathbf{v} + \frac{1}{2}\Delta t, \mathbf{k}_2\right), \mathbf{k}_4 = f\left(\mathbf{p} + \Delta t, \mathbf{k}_3, \mathbf{v} + \Delta t, \mathbf{k}_3\right) \quad (14)$$

with updates

$$\mathbf{p}(t + \Delta t) = \mathbf{p}(t) + \frac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4), \quad (15)$$

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \frac{\Delta t}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4). \quad (16)$$

8 Summary

This methods section provides a unified framework for simulating multi-agent aerial swarms using physically motivated second-order dynamics and structured control laws. The formulation supports multi-group coordination, obstacle avoidance, and mission-level behaviors while remaining computationally efficient and mathematically transparent.